

Project Proposal

Team - 3 Hostel Management System

Team Members

Name	SRN
Shalik S	PES2UG24CS454
Shreyas Patil	PES2UG24CS492
Shreyas Mohanty	PES2UG24CS491
Shobhit Rachit Keshri	PES2UG24CS475

2. Proposed Project Description

Right now, room allocation, fee tracking, and complaints at most college hostels still run on paper registers or spreadsheets nobody quite trusts. We're building a web app to replace that for our own hostel. There's no paying customer here — it's for the students living in the hostel and whichever warden or admin staff run things day to day.

Checking which rooms are free means flipping through a register or an old spreadsheet, and it's easy for two students to end up allotted the same room by mistake. Complaints get raised and then nobody quite remembers to close them out. We want students to handle most of this themselves through a portal, and give the warden one dashboard instead of a stack of registers.

Typical users

- **Students** — apply for a room, view allocation status, pay fees, raise maintenance complaints, log visitors.
- **Warden / Admin** — set up blocks and rooms, allocate and reassign rooms, track fee payments, resolve complaints, view occupancy reports.

Tech stack:

Python 3.x with Django. SQLite while we're developing, PostgreSQL once it's deployed. Front end is just server-rendered Django templates — nothing fancier.

3. Functional Features

Here's what we expect to have working by the time the project is done.

1. **Authentication and role-based access control** — login and logout with two roles, student and admin. The hostel warden logs in as admin.
2. **Student registration and profile management** — a student registers with SRN, name, email, and contact number; an admin can view and correct any student record.
3. **Hostel block and room management** — admin creates blocks and rooms with capacity, room type, and fee for the term.
4. **Room allocation and availability tracking** — admin assigns a student to a vacant room, can reassign or deallocate, and sees a live view of vacant, partially occupied, and full rooms by block and floor.
5. **Fee management** — amount due and amount paid per student, with automatic balance calculation and a printable receipt.
6. **Complaint management** — student raises a maintenance complaint, admin moves it through Open, In Progress, and Resolved.
7. **Visitor log** — record visitor entry, purpose, host student, and check-out time.
8. **Admin dashboard** — occupancy percentage, pending fees, and open complaints at a glance.
9. **Reports generation** — occupancy report, fee-due report, and complaint status report, exportable as CSV.
10. **Search and filter** — find students by name or SRN, and filter rooms and complaints by their attributes.

Note: We kept the same numbering for these ten features in the SRS and in the Functional and Non-Functional Requirements document, where each one gets broken down further into requirements we can actually test.

A few things we're deliberately leaving out of this version: leave/outpass management, an online payment gateway, email or SMS notifications, mess and attendance tracking, and support for more than one hostel. The SRS has the reasoning behind each of those.

4. Plan of Work

Week	Work
1	Finalise the SRS and the database schema
1–2	Set up the Django project skeleton, models, and authentication
2–3	Build block and room management, then room allocation and availability
3–4	Build the fee and complaint modules
4–5	Build the visitor log, admin dashboard, and reports
5–6	Integration testing, search and filter, and bug fixing

5. Product Ownership

Each of us owns two functional features, plus one quality attribute of the system we're on the hook for.

Member	Functional features owned	Qualitative property owned
Shalik S (Lead)	Authentication and access control (feature 1), student registration and profiles (feature 2), admin dashboard (feature 8)	Authentication must be secure — passwords hashed, and role checks enforced on the server for every admin route, not just hidden in the interface
Shreyas Patil	Block and room management (feature 3), room allocation and availability tracking (feature 4)	Room status must stay correct under concurrent edits — no double-allotment of a single room, and occupancy never exceeding capacity
Shreyas Mohanty	Fee management (feature 5), reports (feature 9)	Fee and occupancy reports must reflect the current database state and generate in under two seconds
Shobhit Rachit Keshri	Complaint management (feature 6), visitor log (feature 7)	Complaint status updates must be visible to the student within one page refresh, with no stale cached value

Search and filter (feature 10) doesn't belong to any one person — whoever owns a module builds the search and filter for it.

Note- Beyond our owned features, we're all pitching in on the shared work too — interface, database design, unit testing. Per the proposal guidelines, that shared work doesn't count toward anyone's two features.